



2026–2027



جب کوئی قوم فن اور علم
سے عاری ہو جاتی ہے
تو وہ غربت کو دعوت
دیتی ہے اور جب غربت
آتی ہے تو وہ ہزاروں
جرائم کو جنم دیتی ہے۔

“When a nation becomes devoid
of art and learning, it invites
poverty and when poverty comes
it brings in its wake thousands of
crimes.”

– Sir Syed Ahmad Khan

Lab-V

Course Code- CABSMJ5P05



B.Sc. (CA) V Semester Lab Manual

DEPARTMENT OF COMPUTER SCIENCE
ALIGARH MUSLIM UNIVERSITY, ALIGARH

2026–2027

Credits

The following lab manual up-gradation committee updated the lab manual:

- ▶ *Prof.Arman Rasool Faridi (Chairperson)*
- ▶ *Prof. Mohammad UbaidUllah Bokhari*
- ▶ *Prof. Aasim Zafar*
- ▶ *Prof. Suhel Mustajab*
- ▶ *Dr. Shafiqul Abidin*
- ▶ *Dr. Asif Irshad Khan*
- ▶ *Dr. Faisal Anwar*
- ▶ *Dr. Mohammad Sajid*
- ▶ *Dr. Mohammad Nadeem*
- ▶ *Dr. Faraz Masood*

The following committee member originally design the lab manual:

- ▶ *Prof. Mohammad Ubaidullah Bokhari*
- ▶ *Dr. Arman Rasool Faridi*
- ▶ *Dr. Faisal Anwer*
- ▶ *Prof. AasimZafar (Converner)*

Design & Compilation:

- ▶ *Dr. Faraz Masood*

Revised Edition: July, 2026

Department of Computer Science, A.M.U., Aligarh (U.P.) India

Lab Manual: Lab – V (CABSMJ5P05)

Course Description	3
Content	3
Objectives	4
Outcomes	5
Rules & Regulations	5

COURSE TITLE:	Lab-V	COURSE CODE:	CABSMJ5P05
CREDIT:	4	PERIODS PER WEEK:	6
CONTINUOUS ASSESSMENT:	60	EXAMS:	40

COURSE DESCRIPTION

This course is designed to provide students with a comprehensive understanding of fundamental programming principles, data structures, and system-level operations. This course will guide students through essential programming constructs, including arithmetic operations, loops, conditionals, and functions, while also introducing them to more advanced concepts such as object-oriented design and implementation. Students will engage in hands-on exercises involving arrays, matrices, stacks, and queues, and will apply algorithms for sorting, searching, and other computational tasks. The course also covers critical system-level topics, including shell scripting, file management, process scheduling, and deadlock avoidance, offering a practical understanding of how operating systems function. Additionally, students will enhance their logical and mathematical reasoning skills by tackling problems related to number systems, prime number identification, sequence generation, and other mathematical operations. By the end of the course, students will have gained the knowledge and skills necessary to solve a wide range of programming challenges and to understand the underlying mechanisms of computer systems.

CONTENT

The lab manual is designed to equip students with a solid foundation in fundamental programming techniques, including arithmetic operations, conditional statements, loops, and functions. Through practical exercises, students will develop skills in working with essential data structures such as arrays, matrices, stacks, and queues, and apply algorithms for searching, sorting, and other operations. The manual also introduces object-oriented principles, teaching students to design and implement systems using concepts like inheritance, polymorphism, and design patterns, complemented by UML diagrams. Additionally, students will gain proficiency in system-level programming by writing shell scripts and C programs that handle tasks such as file management, process scheduling, and deadlock avoidance. Finally, the manual emphasizes the application of logical and mathematical reasoning in programming, guiding students through challenges involving number system conversions, prime checks, sequence generation, and other computational problems.

OBJECTIVES

This course is designed to help students in:

- ▶ **To Master Fundamental Programming Techniques:** Students will learn to implement basic programming constructs, including arithmetic operations, conditional statements, loops, and functions, in order to solve fundamental computational problems effectively.
- ▶ **To Develop Skills in Data Structures and Algorithms:** Students will gain the ability to work with essential data structures such as arrays, matrices, stacks, and queues, and apply algorithms for searching, sorting, and other operations that are crucial in computer science.
- ▶ **To Understand and Apply Object-Oriented Principles:** Students will learn to design and implement object-oriented systems by understanding and applying concepts such as inheritance, polymorphism, and design patterns, and by creating UML diagrams to represent these designs.
- ▶ **To Acquire System-Level Programming Proficiency:** Students will become proficient in system-level programming by writing shell scripts and C programs that involve file management, process scheduling, deadlock handling, and other operating system concepts.
- ▶ **To Apply Logical and Mathematical Reasoning in Programming:** Students will develop the ability to solve problems involving number system conversions, prime number checks, sequence generation, and other mathematical operations, using logical and computational thinking in their programming tasks.

OUTCOMES

After completing this course, the students would be able to:

- ▶ **Proficiency in Basic Programming Constructs:** Students will develop a strong foundation in programming by mastering essential constructs such as arithmetic operations, conditionals, loops, and functions. They will be able to apply these concepts to solve a variety of basic computational problems.
- ▶ **Ability to Work with Data Structures and Algorithms:** Through exercises involving arrays, matrices, and data structures like stacks and queues, students will gain hands-on experience in implementing and manipulating these structures, as well as applying algorithms for sorting, searching, and mathematical operations.
- ▶ **Understanding of Object-Oriented Design and Implementation:** Students will learn to design and implement object-oriented systems, utilizing concepts like inheritance, polymorphism, and design patterns. They will be able to translate design into code using UML diagrams and simulate object-oriented models in C++ or Java.
- ▶ **Competence in System-Level Programming:** By working on shell scripts and system-level programs, students will become familiar with file management, process scheduling, and deadlock handling in operating systems. They will also gain experience in simulating scheduling algorithms and other system-level tasks in C.
- ▶ **Capability to Apply Logical and Mathematical Concepts:** Students will enhance their problem-solving skills by implementing programs that involve number system conversions, prime checks, Fibonacci sequences, and palindrome detection. They will be able to apply logical thinking to a wide range of mathematical and computational challenges.

RULES AND REGULATIONS

Students are required to strictly adhere to the following rules.

- ▶ The students must complete the weekly activities/assignments well in time (i.e., within the same week).
- ▶ The students must maintain the Lab File of their completed problems in the prescribed format (Appendix-1).

RULES AND REGULATIONS

- ▶ The students must get the completed weekly problems checked and signed by the concerned teachers in the Lab in the immediate succeeding week. Failing which the activities/assignments for that week will be treated as incomplete.
- ▶ At least TEN (10) such timely completed and duly signed weekly activities/assignments are compulsory, failing which students will not be allowed to appear in the final Lab Examination.
- ▶ Late submission would not be accepted after the due date.
- ▶ Cooperate, collaborate and explore for the best individual learning outcomes but copying is strictly prohibited.

APPENDIX–I**Template for the Index of Lab File**

WEEK NO.	PROBLEMS WITH DESCRIPTION	PAGE NO.	SIGNATURE OF THE TEACHER WITH DATE
1	1#		
	2#		
	3#		
2	1#		
	2#		
	3#		
3	1#		
	2#		
	3#		

Note: The students should use Header and Footer mentioning their roll no. & name in footer and page no in header.

WEEK #1**OBJECTIVES**

- ▶ To introduce OS-related basic programming by working with process IDs and CPU time concepts.
- ▶ To develop logic-building skills required for OS resource calculations.

OUTCOMES

After completing this week, the students would be able to:

- ▶ Students will be able to write basic programs involving time conversion, process categorization, and resource distribution.
- ▶ Students will understand simple OS-related terms like CPU burst time and context switching.

Problems

- 1. CPU Time Conversion:** Convert CPU burst time (seconds) into HH:MM:SS.
- 2. Process Type Identification:** Input a process ID and print "System Process" if PID is even, else "User Process."
- 3. Context Switch Overhead:** Given total processes and context switch time, calculate total overhead.
- 4. Resource Distribution:** Given total printers and users, calculate how many printers each user can get equally.

WEEK #2**OBJECTIVES**

- ▶ To understand fundamental OS performance metrics like CPU utilization and throughput.
- ▶ To practice logical decision-making based on priority levels.

OUTCOMES

After completing this week, the students would be able to:

- ▶ Students will calculate basic OS performance measures.
- ▶ Students will gain an initial understanding of priority-based execution.

PROBLEMS

1. **CPU Utilization:** Calculate CPU utilization given busy_time and idle_time.
2. **Priority Selection:** Input priorities of 5 processes (1=High, 2=Medium, 3=Low) and print which process runs first.
3. **Throughput Calculation:** Given n processes and total time, calculate throughput (processes/time).
4. **Ready Queue Order:** Take process names as input and display them in the order they will execute (manual ready queue).

WEEK #3**OBJECTIVES**

- ▶ To store and manipulate process-related data using arrays.
- ▶ To develop logic for selecting processes based on CPU burst times.

OUTCOMES

After completing this week, the students would be able to:

- ▶ Students will be able to calculate average burst times and sort processes.
- ▶ Students will gain practical experience in process selection logic.

PROBLEMS

1. **Burst Time Average:** Input burst times for n processes and calculate total and average burst time.
2. **Shortest Process Finder:** Find and display the process with the shortest burst time.
3. **Sort by Arrival Time:** Sort processes based on arrival times in ascending order.
4. **Ready Queue Simulation:** Input process names and store them in a ready queue.

WEEK #4**OBJECTIVES**

- ▶ To represent process information using C structures.
- ▶ To calculate process completion times.

OUTCOMES

After completing this week, the students would be able to:

- ▶ Students will be able to store, display, and calculate completion times for processes.

PROBLEMS

1. **Define Process Structure:** Create a structure Process with pid, arrival_time, burst_time.
2. **Input & Display Processes:** Write functions to input and display processes.
3. **Completion Time Calculation:** Calculate $\text{completion_time} = \text{arrival_time} + \text{burst_time}$ for each process.
4. **Next Process Selection (SJN):** Find the process with the shortest burst time from the ready queue.

WEEK #5**OBJECTIVES**

- ▶ To calculate waiting and turnaround times for processes.
- ▶ To understand Gantt charts.

OUTCOMES

After completing this week, the students would be able to:

- ▶ Students will manually calculate basic scheduling parameters.
- ▶ Students will understand how to represent execution order.

PROBLEMS

1. **Turnaround & Waiting Time:** Calculate turnaround and waiting times for given processes.
2. **Gantt Chart Representation:** Display a textual Gantt chart timeline for 3 processes.
3. **Response Ratio Calculation:** Calculate response ratio for each process.
4. **Arrival-Based Ready Queue:** Simulate a ready queue considering arrival times.

WEEK #6**OBJECTIVES**

- ▶ To implement the queue data structure.
- ▶ To implement FCFS and SJN scheduling algorithms.

OUTCOMES

After completing this week, the students would be able to:

- ▶ Students will implement basic scheduling algorithms.
- ▶ Students will understand ready queue operations.

PROBLEMS

1. **Queue Implementation:** Enqueue, dequeue, and display operations.
2. **FCFS Scheduling (C):** Complete FCFS with average waiting & turnaround time.
3. **SJN Scheduling (C):** Implement SJN (non-preemptive).
4. **Shell Script:** Convert CPU burst time from ms to ns, μ s, and s.

WEEK #7**OBJECTIVES**

- ▶ To learn and implement preemptive scheduling algorithms.
- ▶ To calculate starvation in priority scheduling.

OUTCOMES

After completing this week, the students would be able to:

- ▶ Students will simulate Round Robin and Priority scheduling.
- ▶ Students will analyze starvation issues.

PROBLEMS

1. **Round Robin Scheduling:** Implement RR with time quantum.
2. **Priority Scheduling:** Implement priority scheduling.
3. **Starvation Time Calculation:** Calculate starvation time for each process.
4. **Shell Script:** Display system uptime and running processes.
5. **OOAD:** Create a UML class diagram for a simple system of your choice. Add in report how analysis was done. (In Visual Paradigm)

WEEK #8**OBJECTIVES**

- ▶ To understand deadlocks and avoidance techniques.
- ▶ To implement Banker's Algorithm.

OUTCOMES

After completing this week, the students would be able to:

- ▶ Students will detect and avoid deadlocks using simple algorithms.

PROBLEMS

1. **Deadlock Detection:** Check for circular wait using simple arrays.
2. **Banker's Algorithm:** Implement Banker's Algorithm for deadlock avoidance.
3. **Resource Request Safety Check:** Check if request can be granted safely.
4. **Shell Script:** List running processes using more than 5% CPU.
5. **OOAD:** Create a UML object diagram for the same system as in previous week.

WEEK #9**OBJECTIVES**

- ▶ To understand paging and basic memory allocation strategies.

OUTCOMES

After completing this week, the students would be able to:

- ▶ Students will calculate page numbers, offsets, and simulate memory allocation.

PROBLEMS

1. **Paging Simulation:** Find page number & offset for given logical addresses.
2. **First-Fit Allocation:** Allocate memory blocks using First-Fit.
3. **Best-Fit & Worst-Fit Comparison:** Compare fragmentation in both.
4. **Shell Script:** Display top 5 memory-consuming processes.
5. **OOAD:** Create a UML activity diagram for the same system as in previous week.

WEEK #10**OBJECTIVES**

- ▶ To perform file handling operations using C.
- ▶ To write and read process information from files.

OUTCOMES

After completing this week, the students would be able to:

- ▶ Students will store and retrieve process data in files.

PROBLEMS

1. **Create Process File:** Store PID & burst time in a file.
2. **Read Process File:** Display stored process information.
3. **Count Processes in File:** Count the number of entries.
4. **Shell Script:** Display file size, permissions, and last modified date.
5. **OOAD:** Create a UML sequence diagram for the same system as in previous week.

WEEK #11**OBJECTIVES**

- ▶ To understand disk scheduling techniques.

OUTCOMES

After completing this week, the students would be able to:

- ▶ Students will calculate seek time for different disk scheduling algorithms.

PROBLEMS

1. **FCFS Disk Scheduling:** Calculate total head movement.
2. **SSTF Scheduling:** Compare seek time with FCFS.
3. **SCAN & C-SCAN:** Implement both and compare results.
4. **Shell Script:** Display disk space usage.
5. **OOAD:** Create a UML package diagram for the same system as in previous week.

WEEK #12**OBJECTIVES**

- ▶ To simulate process synchronization problems.

OUTCOMES

After completing this week, the students would be able to:

- ▶ Students will simulate producer-consumer and critical section problems.

PROBLEMS

1. **Producer-Consumer Problem:** Simulate using arrays and semaphores.
2. **Dining Philosophers Problem:** Simulate using array-based logic.
3. **Readers-Writers Problem:** Simple simulation without threads.
4. **Shell Script:** Auto-refresh list of top 5 CPU-consuming processes.

WEEK #13**OBJECTIVES**

- ▶ To simulate page replacement and file allocation techniques.

OUTCOMES

After completing this week, the students would be able to:

- ▶ Students will understand page faults and file allocation methods.

PROBLEMS

1. **Page Replacement (FIFO & LRU):** Simulate and calculate page faults.
2. **Indexed File Allocation:** Simulate basic indexed file allocation.
3. **Contiguous vs Linked Allocation:** Simulate both methods.
4. **Shell Script:** Search and list files >100MB.

WEEK #14**OBJECTIVES**

- ▶ To integrate multiple OS concepts into a single simulation.

OUTCOMES

After completing this week, the students would be able to:

- ▶ Students will design a complete OS simulation as a mini-project.

PROBLEMS

1. **Menu-driven Process Scheduler:** Combine FCFS, SJN, RR, and Priority scheduling.
2. **Deadlock Detector with Banker's Algorithm:** Dynamic resource request handling.
3. **Memory Management Visualizer:** Simulate First-Fit, Best-Fit, and fragmentation report.
4. **Shell Script – OS Health Monitor:** Combined CPU, memory, and disk report.